Reviewer Pack — Minimal Order of Twisted Module Representations and Frege Pr...

Entry b12cfc99fd12 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-02 09:26:03 UTC

Minimal Order of Twisted Module Representations and Frege Proof Length Correlation

Entry ID: `b12cfc99fd12` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-02 09:26:03 UTC

1. Conjecture

Field A (mathematical branch): Twisted Module Theory **Field B** (complexity object): Frege Proof Complexity

Statement:

> For every k-regular graph G, the minimal order of twisted module representations associated with its associated formula φ_G is linearly correlated with its Frege proof length l(φ_G), such that the order = Θ(l(φ_G))

Rationale (proposer's reasoning):

> Twisted modules provide a way to encode combinatorial data in a structured manner that might capture the complexity of proofs. Their algebraic nature could reveal hidden structures within formulas, leading to insights into proof complexity.

Taxonomy category: `Twisted_Module_Theory` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `f93b1fa9ba58993b`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> For a given k-regular graph G, if the correlation coefficient between the minimal order of twisted module representations (order) and Frege proof length (l) is ≥ 0.8 for all graphs with n ≤ 40, then the conjecture is supported.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS

Barrier	Verdict	Confidence	LLM₁	LLM₂
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: `ADJACENT_OK` against 1 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `"twisted module theory" AND "Frege proof complexity"` - `"minimal order twisted module representations" AND "Frege proof length"` - `"correlation k-regular graph minimal order" AND "linear Frege proof length"`

Top relevant hits considered: - [s2:2101.12287] Support varieties over skew complete intersections via derived braided Hochschild cohomology

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_k_regular_graph(n, k):
        if (n * k) % 2 != 0:
            return None
        adj_matrix = [[0] * n for _ in range(n)]
        edges_added = 0

        while edges_added < k * n // 2:
            u, v = random.sample(range(n), 2)
            if adj_matrix[u][v] == 0 and u != v:
                adj_matrix[u][v] = 1
                adj_matrix[v][u] = 1
                edges_added += 1

        return adj_matrix

    def tseitin_encoding(graph):
        n = len(graph)
        literals = list(range(1, 2 * n + 1))
        clauses = []

        for i in range(n):
            clause = [literals[2 * i], literals[2 * i + 1]]
            clauses.append(clause)

        for u in range(n):
```

```
            for v in range(u + 1, n):
                if graph[u][v] == 1:
                    neg_u = -literals[2 * u]
                    neg_v = -literals[2 * v]
                    clause = [neg_u, neg_v, literals[n * 2]]
                    clauses.append(clause)

        return clauses

    def minimal_twisted_module_order(clauses):
        n = len(clauses)
        if not clauses:
            return 0

        order = 1
        for i in range(1, n + 1):
            if all(literal in clause for literal in range(-i, i + 1)):
                order = i
                break

        return order

    def frege_proof_length(clauses):
        # Placeholder function to simulate Frege proof length calculation
        # This is a dummy implementation and should be replaced with actual logic
        return len(clauses)

    n_max = 40
    instances_tested = 0
    correlation_coefficient = 0.0

    for n in range(5, 41):
        graph = generate_k_regular_graph(n, random.randint(2, min(n - 1, 3)))
        if graph is None:
            continue

        clauses = tseitin_encoding(graph)
        if not clauses:
            continue

        order = minimal_twisted_module_order(clauses)
        proof_length = frege_proof_length(clauses)

        correlation_coefficient += (order * proof_length) / n
        instances_tested += 1

    if instances_tested == 0:
        return {
            "metric_name": "correlation_coefficient",
            "metric_value": None,
            "instances_tested": 0,
            "n_max": 0,
            "conjecture_holds": False,
            "counterexample": "mapping_undefined"
        }
```

```
    correlation_coefficient /= instances_tested

    return {
        "metric_name": "correlation_coefficient",
        "metric_value": correlation_coefficient,
        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": correlation_coefficient >= 0.8,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(arg) for arg in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41,

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{'seed': {seed}, ...run_trial output...}}")
        results.append(result)

    mean_value = sum(result["metric_value"] for result in results if result["metric_value"] is not
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std=0.0 support_fraction={support_fraction}")
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample='' first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a2da8e72.py", line 119, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a2da8e72.py", line 82, in run_trial
    clauses = tseitin_encoding(graph)
              ^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a2da8e72.py", line 50, in tseitin_encod
    clause = [neg_u, neg_v, literals[n * 2]]
                            ~~~~~~~~^^^^^^^
IndexError: list index out of range
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test code crashed before producing data, which means that it was unable to compute the correlation coefficient required to evaluate the conjecture. | next: Investigate and fix the crash in the test code to allow for the computation of the correlation coefficient.

11. Audit log (LLM calls)

Total LLM calls: 11

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	16916
2	propose	ollama_remote	glm4:latest	0	0	13893
3	propose	ollama_remote	glm4:latest	0	0	15486
4	preregistration	ollama_remote	glm4:latest	0	0	14976
5	novelty	ollama_remote	glm4:latest	0	0	18159
6	novelty	ollama_remote	glm4:latest	0	0	9463
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12978
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9543
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16868
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18875
11	judge	ollama_remote	glm4:latest	0	0	8980

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 156138 ms total latency. Provider mix: {'ollama_remote': 11}

(full prompt+response transcripts available in `research/audit/b12cfc99fd12.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/b12cfc99fd12.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/b12cfc99fd12.tar.gz` (if generated)